Harnessing WebRTC for Large-Scale Live Streaming

Wei Zhang[†], Tong Meng[†*], Xianhua Zeng[†], Wei Yang[†], Changqing Yan[†], Chao Li[†]
Chenguang Li[†], Feng Qian[†], Junfeng Yang[‡], Lei Zhang[§], Zhi Wang[¶]
[†]ByteDance [‡]Hunan University of Technology and Business [§]Shenzhen University
[¶]SIGS, Tsinghua University

Abstract

Live streaming that supports real-time interaction has become increasingly popular. To support the ensuing requirements on low end-to-end latency, RTM, the state-of-the-art live streaming system at Douyin, replaces the HTTP-FLV streaming protocol with WebRTC. To tailor the WebRTC stack to the live streaming scenario, we focus on optimizing first-frame delay, startup video rebuffering, audio-to-video drift, and per-session CPU usage. Those are the top-priority metrics identified from an importance analysis with respect to two user engagement metrics, *i.e.*, viewer penetration and viewing time. To date, WebRTC-based streaming in RTM has been in operation for 4 years, and serves billions of viewer sessions every day. It dramatically optimizes QoE metrics (*e.g.*, end-to-end latency reduced by 54.5%), and delivers statistically significant user engagement gains (*e.g.*, number of paid orders increased by 0.8%). In this paper, we report our deployment experiences comprehensively.

CCS Concepts

• **Networks → Application layer protocols**; • **Information systems → Data mining**; **Multimedia streaming**.

Keywords

WebRTC, HTTP-FLV, Live Streaming, QoE, User Engagement, AV Drift, CPU Usage

ACM Reference Format:
Wei Zhang, Tong Meng, Xianhua Zeng, Wei Yang, Changqing Yan, Chao Li, Chenguang Li, Feng Qian, Junfeng Yang, Lei Zhang, Zhi Wang. 2025. Harnessing WebRTC for Large-Scale Live Streaming. In *ACM SIGCOMM 2025 Conference (SIGCOMM '25), September 8–11, 2025, Coimbra, Portugal.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3718958.3750535

1 Introduction

Live streaming has reshaped how people connect with each other. Live media contents shared on social media platforms (*e.g.*, Twitch [4], YouTube Live [7], *etc.*) reach one-third of Internet users, and contribute 17% of total Internet traffic [48]. Many emerging use cases (*e.g.*, e-commerce, live sports, *etc.*) encourage interaction between the broadcaster and viewers, and urge low end-to-end streaming latency. For example, online merchants need a short streaming delay for their consumers while holding "first-comment-to-get" flash sale activities. Football fans get frustrated if they always get spoilers from the chat board about upcoming goals before seeing them in the live video stream.

Douyin [2], as one of the largest live streaming platforms in China, serves billions of viewer sessions daily. Over the past years, our live streaming architecture has been based on RTMP ingest from the broadcasters and HTTP-FLV streaming to the viewers. After years of optimization on edge CDN deployment and overlay routing, we managed to achieve an end-to-end latency (*i.e.*, time behind live [50]) below 5 seconds. However, that may still be inadequate in some near real-time scenarios. Through a breakdown of each segment during end-to-end transmission, we found that the distribution side (*i.e.*, from CDN to viewer) dominates the streaming latency. Therein, several seconds of video frames should be pre-buffered before starting playback, to resist fluctuation on access links. Thus, among various components in our system, we expect streaming protocol optimization to yield the most significant benefit.

It may appear straightforward to lower latency by relaxing pre-buffering. Yet considering heterogeneous access network conditions at different viewers, that is very likely to bring negative side effects. In the process of pushing our system from 5∼10 seconds to 3∼5 seconds, we tried directly reducing pre-buffering from 6 seconds to 3 seconds. That did reduce the end-to-end latency, but came with dramatic increments in rebuffering, as well. It took us months to remedy such degradation, *e.g.*, by adaptive pre-buffering according to viewers' access link conditions.

As we approach the limit of the underlying TCP transport, we determined to optimize the end-to-end latency in a more fundamental way. That is, to leverage the WebRTC stack and replace the HTTP-FLV streaming protocol with RTP [46]. Aimed at real-time communication (RTC), WebRTC is able to achieve sub-second latency. It encompasses a suite of features, including but not limited to customized congestion control [19, 29] and jitter buffer management [23, 60], enabling robust performance in fluctuating networks. More importantly, it is already supported by some pioneering CDN and cloud providers [24, 33].

As expected, simply deploying the WebRTC implementation as in our RTC applications at ByteDance[1] lowered the end-to-end latency by nearly 50%. However, without further optimization, the vanilla WebRTC stack impacted our user engagement, *e.g.*, the average daily viewing time per user was reduced by 1.28%. In-depth analysis shows that some features in WebRTC are not well-tailored to the

[*]Tong Meng is the corresponding author: mengtong.mt@bytedance.com

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACM SIGCOMM'25, Coimbra, Portugal
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1524-2/2025/09
https://doi.org/10.1145/3718958.3750535

[1]ByteDance serves various applications besides Douyin, such as Lark for multi-party video conferencing, Doubao as a general-purpose AI chatbot.

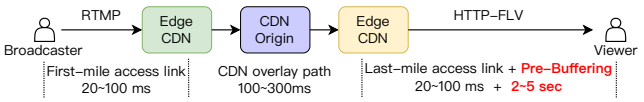


Figure 1: End-to-end streaming latency breakdown

live streaming scenario. For instance, an RTC applications may accomplish AV (audio-to-video) synchronization and lower end-to-end latency by dropping delayed video frames. However, according to our experiences in operating HTTP-FLV, such a strategy does not quite align with the needs of live streaming viewers, who tend to have higher requirements for rendering smoothness.

Therefore, it is a non-trivial task to tame WebRTC as a live streaming protocol. To be cost-effective, we first identify metrics with the highest optimization priority. For that purpose, we decide on two user engagement metrics that we care the most, *i.e.*, viewer penetration[2] and daily viewing time[3]. The other metrics are sorted based on their importance relative to user engagement. On one hand, the two user engagement metrics directly reflect user satisfaction, and can largely determine our revenue from the application. They are suitable as our ultimate optimization goals. On the other hand, any modification to the streaming protocol influences user engagement indirectly through other QoE metrics. The QoE metrics are what we can directly optimize.

Then, we propose mechanisms to optimize the four identified top-priority metrics in the previous step. More specifically, we implement an integrated media processing pipeline to lower viewer session CPU usage, as well as first-frame delay and startup rebuffering; we leverage audio-calibrated AV synchronization to reduce AV drift; we mitigate startup rebuffering by bitrate-limited startup pacing.

The above efforts are all part of the construction of RTM (Real-Time Media), the new live streaming system of Douyin. RTM evolves both RTMP ingest and HTTP-FLV streaming to the WebRTC stack, and this paper only focuses on the downlink streaming protocol. We should note that RTM is not possible without the support of WebRTC-based live streaming by CDN providers [33]. Still, to the best of our knowledge, we are the first to share practical experiences on tailoring WebRTC as a live streaming protocol from the perspective of a billion-user application service provider. In addition to re-confirming its benefit of lower end-to-end latency, we report our journey of optimizing the stack to finally boost user engagement as well as commercial revenue. Our main contribution is highlighted as follows.

- We identify top-priority metrics by analyzing the importance of various QoE metrics on two carefully selected user engagement metrics, *i.e.*, viewer penetration and viewing time. Four metrics are determined as our optimization objectives, which are first-frame delay, startup video rebuffering count, AV drift, and CPU usage.
- We tailor RTM to live streaming scenarios by various optimization mechanisms aimed at the top-priority metrics, including an integrated media processing pipeline, audio-calibrated AV sync, and bitrate-limited startup pacing.
- We verify the effectiveness of the importance analysis and the proposed optimization mechanisms through multiple A/B tests involving billions of viewer sessions. Practical deployment shows that RTM significantly outperforms HTTP-FLV, *e.g.*, end-to-end latency is reduced by 54.5%, rebuffering is reduced by up to more than 70%. As a result, our user engagement sees statistically significant increments, *e.g.*, viewer penetration and daily viewing time are increased by 0.03% and 0.39%, leading to 0.8% more paid orders from live streaming viewers.

This work does not raise any ethical issues.

[2]Viewer penetration is the ratio between the number of times viewers actually enter a live broadcast room and the number of times viewers swipe to a live broadcast.
[3]Daily viewing time is defined as the average duration each viewer stays in any broadcast room each day.

Table 1: Reducing pre-buffering duration from 6 sec to 3 sec causes degradation to rebuffering metrics

Metrics	Average Δ%
End-to-end delay	−39.3%
Video stall duration	+71.6%
Video stall count	+82.0%
Stall session ratio	+69.7%

2 Background and Motivation

2.1 End-to-End Latency Breakdown

To localize the most critical component for optimization in our system, we break down the typical delay of each segment in end-to-end transmission in Figure 1. We focus on network-related delays, and codec processing delay is not counted.

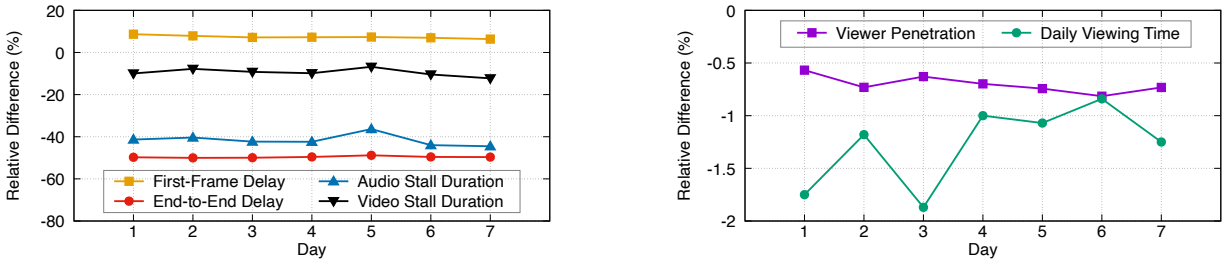
With widely deployed edge CDN PoPs (Points of Presence) nowadays, the average first/last-mile access link delay has been reduced to tens of milliseconds [25]. Meanwhile, the CDN overlay path consumes much less than 300 ms in the majority of cases. That is even lower than the 400 ms norm raised in [33], since most of our clients come from China and do not require cross-continent paths. That leaves viewer-side pre-buffering accounting for nearly 90% of the end-to-end latency. Our HTTP-FLV implementation requires the viewer, as well as the edge CDN server, to pre-buffer video frames of several seconds before starting the playback. That is a common technique to reduce rebuffering against frequent last-mile network fluctuations. Similar mechanisms also exist in other TCP-based streaming protocols such as HLS [12].

Obviously, last-mile latency constitutes the largest proportion among all segments. Correspondingly, we prioritize the optimization of streaming protocol while building RTM.

2.2 Tune Pre-Buffer: Complex than it Seems

Given the breakdown analysis in §2.1, a seemingly simple method to lower the end-to-end latency is to reduce the amount of pre-buffering. Unfortunately, considering the trade-off interaction between different metrics, it cannot unilaterally lower latency without impacting other metrics, especially video and audio rebuffering.

To illustrate that, we present in Table 1 the results of an online A/B test conducted in the initial phase of optimizing HTTP-FLV, where we only changed the configured pre-buffering duration from 6 seconds to 3 seconds. Unsurprisingly, end-to-end latency was reduced by 39.3%. However, that came at the cost of impaired resistance against network fluctuations, leading to significantly more



(a) QoE metrics (b) User engagement

Figure 2: Differences of Vanilla-RTM over HTTP-FLV (Only representative metrics are depicted for brevity.)

rebuffering. Each viewer session experienced 71.6% longer video rebuffering and 82.0% more instances of video rebuffering. There were 69.7% more sessions experiencing audio and video stalls.

Although in the end, we still managed to optimize the end-to-end latency of HTTP-FLV to below 5 seconds without apparent degradation in other metrics, that was made possible by a series of non-trivial mechanisms, *e.g.*, adapting the duration of pre-buffering according to the measured last-mile access link conditions. What is worse, after years of continuous optimization, we started to approach the limit of the TCP-based HTTP-FLV streaming protocol. It costs us exponentially increasing efforts for progressively more marginal performance gains. Therefore, we are motivated to upgrade the streaming protocol more fundamentally.

2.3 Vanilla WebRTC: not a Silver Bullet

The WebRTC protocol stack, with its ability to deliver sub-second level end-to-end latency, is a promising option for our purpose. To consolidate it as the foundation for RTM and to motivate further optimization, we elaborate on observations from a preliminary implementation of RTP-based streaming to viewers (namely vanilla-RTM).

2.3.1 What is in Vanilla-RTM. Thanks to our extensive experience in serving RTC applications at ByteDance (*e.g.*, multi-party conferencing in Lark [3]), we did not need to build RTM based on a general-purpose open-source codebase [6] from scratch. Although named Vanilla-RTM, it encompassed a mature set of basic WebRTC functions tailored to our server-side implementation, *e.g.*, ICE connection establishment [30], customized SDP negotiation [15], *etc.*Such a suite of basic protocols and standards in WebRTC are inherently generalizable to other use cases. In addition, Vanilla-RTM adopted optimized congestion control algorithms specifically designed for live media, as well. For example, the congestion controller on our self-constructed CDN servers is originally based on BBR [18], and then tailored to our requirements. It includes an advanced mechanism that differentiates between random packet loss and congestion loss, enabling more accurate bandwidth estimation.

2.3.2 Deployment Results. We compared Vanilla-RTM and HTTP-FLV through a one-week A/B test involving 450 million viewer sessions for each of them. Compared to HTTP-FLV, Vanilla-RTM achieved improvements on most QoE metrics. As exemplified in Figure 2(a), the average end-to-end latency was reduced by 49.6%, and the duration of audio and video stalls per session was reduced by 41.6% and 9.5%, respectively. One of the most important metrics that saw apparent degradation was the first-frame delay, which was increased by 7.4% on average. However, Vanilla-RTM slightly decreased user engagement as shown in Figure 2(b). Viewer penetration and daily viewing time (metrics defined in §3) declined by 0.70% and 1.28%. Based on our experiences, the two user engagement metrics are crucial to our user satisfaction and their willingness to make in-app payments (*e.g.*, to buy from online shops and to tip the broadcaster). Therefore, further optimization is necessary to fully realize the potential of RTP-based streaming to viewers.

3 Identify Top-Priority QoE Metrics

To improve cost-effectiveness and avoid missing some less recognized but important metrics, we identify the top-priority metrics to optimize according to their estimated importance to two user engagement metrics, *i.e.*, viewer penetration and daily viewing time. In this section, we first explain why we select penetration and viewing time for importance estimation, and then reveal the estimation results.

3.1 User Engagement as Ultimate Objective

We answer two questions to justify our selection of the above two metrics as the ultimate objective.

3.1.1 Why User Engagement? User engagement intuitively reflects users' willingness to use the provided service, and is undoubtedly among the foremost metrics for all service providers. It has two properties that make it suitable as the ultimate optimization objective.

- Usually, user engagement cannot be directly optimized by network-related technical modifications (*e.g.*, the streaming protocol). Instead, they are indirectly influenced by a wide range of metrics (*e.g.*, end-to-end delay). The relative importance of such influence can be deemed a more fair and authoritative evaluation for other metrics than subjective opinions that differ from person to person.
- It is not biased towards any specific user groups or application scenarios. In comparison, in-app payment-related metrics, although directly determining service revenue and at least equally important as user engagement, cannot represent viewers without an online payment habit and broadcasters who do not accept tips and payments.

3.1.2 Why Two Metrics? In the context of live streaming in Douyin, when viewers swipe a live broadcast to foreground display, they

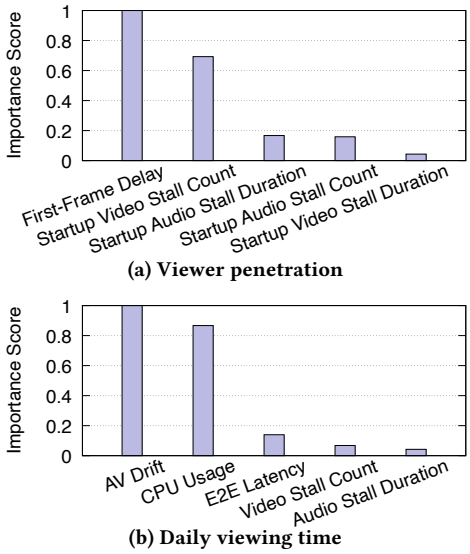


Figure 3: Normalized importance wrt. user engagement

may either swipe it away or click to enter the broadcast room. Viewer penetration and daily viewing time reflect viewer behaviors before and after entering a broadcast room, respectively. Neither of them can deliver a full picture of user engagement on its own. It is possible that viewer penetration is improved while viewing time decreases dramatically, *e.g.*, if we aggressively trade-off rebuffering for faster first-frame delivery. Similarly, when a long end-to-end latency dissuades many impatient viewers from entering live broadcast rooms, the remaining loyal viewers may otherwise pull up the viewing time globally along with a much lower penetration. In both cases, we are in fact losing viewers or/and opportunities to trigger in-app payments. Selecting the two metrics at the same time prevents such local optima.

3.2 Top-Priority Metrics

To quantify the importance of various QoE metrics on user engagement, we leverage XGBoost [21] to train two separate models that predict viewer penetration and daily viewing time, respectively, based on the input of a comprehensive set of QoE metrics. The training set contains data collected from 150 million live streaming sessions under Vanilla-RTM. While training, each model will generate an importance score for each input QoE metric, reflecting its relative influence on the user engagement metric predicted by the model. To enhance the accuracy of the importance estimation, we further interpret the XGBoost models with SHAP [47], and sort the final importance scores.

Figure 3 lists the top-5 QoE metrics specific to viewer penetration and viewing time. For each user engagement metric, there are 2 QoE metrics with obviously leading importance.

First-frame delay. It measures the latency from the time the viewer swipes to a live broadcast to the time the first video frame is rendered. It is not the same as end-to-end delay, *e.g.*, it does not contain the broadcaster-side delay on the first-mile link, but includes additional time initializing the media engine. It is not hard to understand its influence on viewer penetration. An impatient viewer tends to swipe away if the foreground display stays black for too long.

Startup video rebuffering count. It is the number of times the video stalls in the starting phase of a viewer session. It should be pointed out that the count and duration of audio and video rebuffering rank as the 2nd to 5th important metrics, while the video rebuffering count scores at least 4× higher than the other three.

AV Drift. It accounts for the relative drift between the rendered audio and video frames. This issue is primarily caused by two categories of factors: 1) Network-related factors: Fluctuations in network conditions can result in gaps in the arrival times of audio and video packets at the viewer. 2) Independent processing pipelines: Audio and video frames are often processed through two separate pipelines (as will be explained in Figure 4), each requiring varying amounts of processing time. When AV Drift occurs, it increases the burden on viewers when they consistently fail to align the heard audio with the video. In that case, they simply abandon the current broadcast, thus impacting viewing time. Meanwhile, it usually takes only several seconds for a viewer to determine whether to swipe away. It is not long enough for AV drift to induce any significant influence. That explains why it is not a top-priority metric with respect to viewer penetration.

CPU usage. A high CPU pressure may lead to many issues, such as battery overheating. An overwhelmed device is also more likely to suffer from occasional or even frequent media rebuffering. The CPU usage is important in (Vanilla-)RTM, because the WebRTC stack is mostly implemented in user space, and naturally requires higher processing overhead than the kernel implementation in HTTP-FLV.

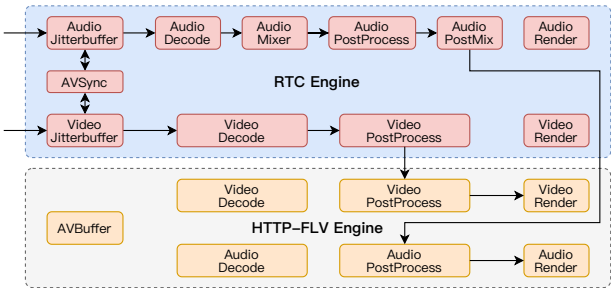
At first sight, the above importance estimation results seem to differ from "common wisdom". For example, end-to-end latency does not have a very high score for both user engagement metrics, and rebuffering-related metrics do not dominate daily viewing time. In fact, that is based on the fact that the WebRTC stack already optimizes the end-to-end latency and rebuffering significantly compared with HTTP-FLV. For other developers in this area, those metrics are still crucial for the live streaming scenario in general.

4 QoE Metrics Optimization

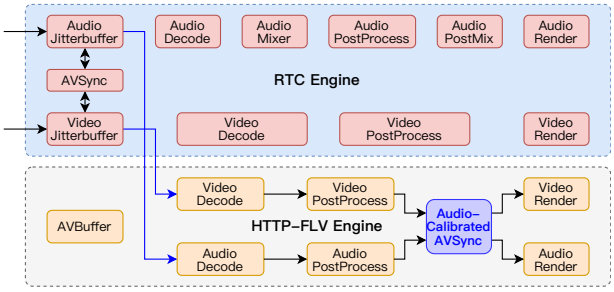
Aimed at the identified top-priority QoE metrics in §3, we delve into several representative optimization mechanisms, among a series of those gradually implemented while we enhance Vanilla-RTM into RTM in its current form over the past 4 years. As a part of our deployment insights, we explain in detail the target problem of each mechanism, and why it is not resolved in Vanilla-RTM.

4.1 Integrated Media Pipeline

4.1.1 Problem: Cumbersome RTC Media Processing Pipeline. In our Vanilla-RTM implementation, we connect the RTC media engine to the HTTP-FLV media engine as shown in Figure 4(a). The jitter buffers in the RTC media engine handle the frame reception and reconstruction. Following that, audio and video frames go through codec-related media processing in the RTC media engine. They are still rendered in the HTTP-FLV media engine, so that the rendering logic is kept the same while viewers swipe between live broadcasts and on-demand videos. However, the almost complete RTC media



(a) Vanilla-RTM



(b) Integrated pipeline in RTM

Figure 4: Viewer-side media processing pipeline

processing pipeline in the RTC media engine brings unnecessary processing overhead.

- First, the life cycle of the codec object in the RTC media engine is bound to the viewer session. Codec initialization happens each time a viewer swipes to a new live broadcast. That raises the CPU consumption in session startup, and delays the first frame, as well.
- Second, many blocks on the RTC audio processing pipeline is in fact not needed for live streaming, and induce additional processing delay. For example, unlike in RTC scenarios where each participant may need to mix multiple audio streams published by other participants, a live streaming broadcast has only a single audio stream and does not require an audio mixer. The post processing updates noise cancellation parameters to minimize the interference from the received audio output to the receiver's own input. Yet the live streaming viewer does not publish any audio stream while watching.

Why not skip RTC codec pipeline? In Vanilla-RTM, the audio and video jitter buffers are implemented with built-in AV synchronization mechanism. It cannot estimate and adjust the target delay of individual frames accurately without the native media processing pipeline in the RTC media engine.[4] The best we could do is to reuse the HTTP-FLV rendering logic, but keep decoding and other processing in the RTC media engine.

Why not a problem in RTC scenarios? Each RTC participant session effectively belongs to an RTC room. The initialization of the codec object can be amortized in the process of joining room, and it saves CPU and memory resources to deconstruct it together with

[4]Interested readers may refer to the WebRTC source code [6] for more AV synchronization details.

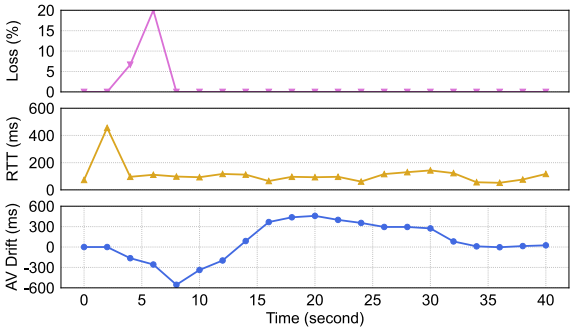


Figure 5: Example of AV drift Problem in WebRTC (A negative drift amount means video falls behind audio)

jitter buffers when the participant leaves the RTC room. Meanwhile, as exemplified above, the audio processing blocks not required in live streaming are all necessary for RTC applications. The corresponding processing delay cannot be avoided.

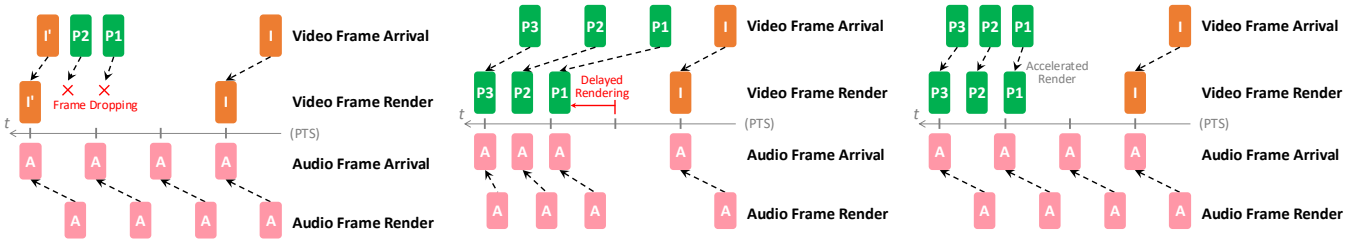
4.1.2 Optimization Solution: Integrated Media Processing Pipeline. We integrate the blocks in the two media engines for improved RTM media processing efficiency. As illustrated in Figure 4(b), the codec object in the HTTP-FLV media engine is now reused by all streaming services in Douyin. When a viewer swipes between on-demand and live streaming media contents, a long-lived codec object is used, without additional CPU or switchover delay due to codec initialization. Along with that, the jitter buffers directly push media frames out of the RTC media engine, saving the overheads of complex RTC-oriented audio processing. Such a simplified pipeline is made possible by our optimized AV synchronization in RTM (with an added sync block in Figure 4(b), as will be explained in §4.2).

4.2 Audio-Calibrated AV Synchronization

4.2.1 Problem: Lack of Strict Synchronization. As explained in §4.1, Vanilla-RTM adopts the same AV synchronization mechanism as in our RTC applications.

Why sync mechanism for RTC scenarios not adequate? In the WebRTC stack, the synchronization module periodically estimates the relative drift amount between the rendered audio and video frames, based on which the target delay of audio and video frames may be adjusted by their respective jitter buffers. Accordingly, a video frame may be delayed by several milliseconds to wait for a congestion-delayed audio frame, and an audio frame may be slightly accelerated to catch up to video rendering. To mitigate the resulting sync-up glitches, such an adjustment happens once every second by default, and each adjustment is bound by 80 ms. A direct consequence of such limitation is the slow sync-up speed. In the worst case, the audio and video signals are consistently out of sync provided a highly fluctuating last-mile access link. For example, Figure 5 showcases an online viewer session. Due to the bursty losses and fluctuating RTTs, it suffers from over 300 ms AV drift for nearly 30 seconds. According to importance analysis in §3, that has significant influence on live streaming user engagement.

Therefore, our RTC media engine also incorporates emergency frame dropping strategies (*e.g.*, usually dropping video frames) to



(a) Frame dropping in Vanilla-RTM (b) Delayed render in HTTP-FLV/RTM (c) Accelerated Render in RTM

Figure 6: Illustrating examples of AV synchronization

speed up sync-up against persistent AV drifts. An example triggering case where video significantly lags behind audio is illustrated in Figure 6(a). When the I-frame of the current GOP is received but considerable P-frames of the previous GOP have not been decoded or rendered, those P-frames will be dropped so that the video rendering can fast forward to the current GOP. Nevertheless, although working well in RTC scenarios, we find that frame dropping is not aligned with live streaming viewer preference.

What we learn from sync mechanism in HTTP-FLV? After years of continuous optimization, HTTP-FLV has learned one most important principle for AV synchronization in the live streaming scenario, *i.e.*, always trying to render audio frames at normal speed, and synchronizing the video up to the audio. For example, an already arrived video frame will not be rendered until the corresponding audio frame is received (as shown in Figure 6(b)). The reason is that audio rebuffering may already happens in that case, and the benefit of avoiding video rebuffering by only rendering video frames is significantly outweighed by its drawback of further increasing the relative AV drift. However, the synchronization mechanism in HTTP-FLV should be based on in-order delivery from the TCP transport and seconds-long pre-buffering. Transplanting it to the WebRTC stack as-is with sub-second-level jitter buffers could easily degrade rebuffering by up to an order of magnitude, especially considering that lowering HTTP-FLV's pre-buffering duration from 6 seconds to 3 seconds almost doubles rebuffering (Table 1).

4.2.2 Optimization Solution: Audio-Calibrated Synchronization. We leverage the basic idea of synchronization mechanism in HTTP-FLV, *i.e.*, to calibrate video rendering to audio rendering. As mentioned in §4.1, an additional sync block is introduced in the HTTP-FLV media engine.

Two-stage synchronization implementation. Based on the integrated media processing pipeline in Figure 4(b), AV synchronization in RTM is accomplished in two stages. The first stage in the RTC media engine is responsible for updating the target delay of media frames. It controls the speed at which jitter buffers push reconstructed frames to the remaining pipeline in the HTTP-FLV media engine. In the second stage, an audio-calibrated synchronization block enforces the rendering time of each frame. It lies in the HTTP-FLV media engine to guarantee precise control, in case the separate video and audio processing pipelines induce further drift. Next, we explain how to determine rendering time of audio and video frames, respectively.

Algorithm 1: Audio-Calibrated Video Rendering

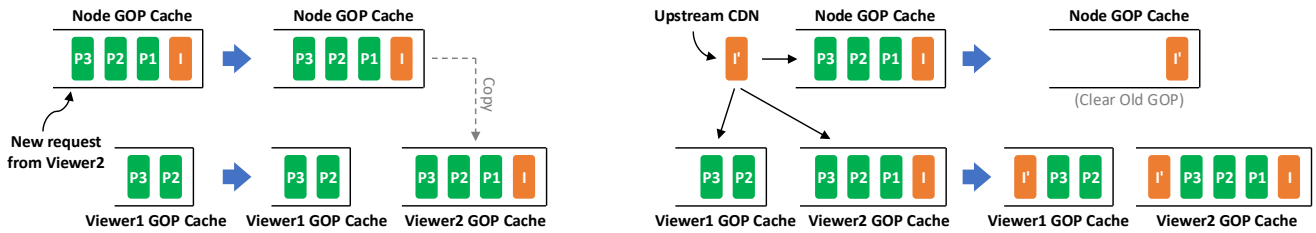
```
// Called when finish rendering each frame
// t_a: current audio rendering progress
// t_v: PTS of next video frame to render
t_a = GetAudioClock(now) ;
if 1st frame of the session then
    Render the frame right now ;
    return
while t_v > t_a + th_a do
    sleep th_a ms ;
    t_a = GetAudioClock(now) ;
Render the frame right now ;
```

Independent audio rendering Audio frames are rendered independently of the reception of video frames. The audio jitter buffer simply tries to align its target delay with reception of recent audio packets (*e.g.*, the 95-th inter-packet arrival jitter). Most of the time, no adjustment is made for the purpose of waiting for or catching up to video rendering. In the second stage, an audio frame will be decoded and rendered right away without further delay.

We note that if video lags behind by a very large amount (*e.g.*, more than 90 ms, above which the AV drift is unacceptable as suggested by [16]) for several seconds, we may temporarily increase target delay of the audio jitter buffer instead of dropping video frames. That is the only scenario the audio target delay is influenced by video rendering.

Audio-calibrated video rendering Every time the rendering of a video frame is complete, the synchronization block in the second stage determines when it can render the next video frame. The audio-calibrated logic is summarized in Algorithm 1. It compares the PTS (presentation time) t_v of the video frame to be rendered, and the current audio rendering clock time t_a, which can be obtained using the PTS of the currently rendered audio frame plus an offset of its rendering progress. The next video frame can only be rendered if t_v is within th_a ahead of t_a. The threshold th_a is set to 25 ms. It is the subjective threshold for AV drift detectability according to the ITU-R BT.1359-1 specification [16]. That is, it becomes undetectable to viewers when audio is advanced with respect to video by no more than 25 ms (th_a).

Therefore, when audio rebuffering occurs and t_a is not updated, the rendering of the next video frame will also be delayed for at least the same amount of time (as in Figure 6(b)). An exception is when it is the first video stream in the session. In that case, we render it



(a) New viewer request (b) New I-frame from upstream CDN

Figure 7: Maintenance of GOP caches at edge CDN server

as soon as possible to avoid impacting first-frame delay and viewer penetration. In the opposite case, when video lags behind, RTM effectively accelerates video rendering as in Figure 6(c) instead of dropping frames as in Vanilla-RTM (Figure 6(a)). In that sense, the calibration is not limited by the periodic synchronization cycle (1 s) and maximum adjustment magnitude (80 ms) as in Vanilla-RTM, and thus, accomplishes sync-up more rapidly.

In addition, we implement a new interface that reports the exact rendering time and relative AV drift from the second-stage sync block to the first-stage block. We also send explicit notifications when 1) the rendering of video frames is paused waiting for a delayed audio frames, and 2) video rendering resumes upon the arrival of the delayed audio frame. Those messages from the HTTP-FLV media engine are used by the RTC media engine to adjust the target delay of video frames. That way, the first-stage AV sync block can properly follow the actual rendering rate. That leads to more accurate target delay adjustment than in Vanilla-RTM, where the calculation of target delay has to rely on estimated decoding and rendering delay.

4.3 Bitrate-Limited Startup Pacing

4.3.1 Problem: Startup Rebuffering due to GOP Cache. When a viewer swipes to a live broadcast, its "current" video frame may not be the key frame in a Group of Pictures (GOP). Without a reference I-frame, this viewer cannot decode the up-to-date video frame. We cannot wait until the next I-frame to start decoding, because that may delay the rendering of the first video frame by over 1 second. Thus, in Vanilla-RTM, we cache the most recent GOP at the edge CDN, and serve new viewer requests from the corresponding cached I-frame. However, the cached frames could easily exceed hundreds of milliseconds when a viewer initiates a new request to a live broadcast. They tend to be transmitted in a large burst or with a very high pacing rate (*e.g.*, due to some very aggressive bandwidth estimation strategy commonly configured at edge CDN), and are prone to last-mile congestion. That ends up introducing more startup video rebuffering.

How HTTP-FLV deals with the problem? HTTP-FLV uses TCP in transport layer. It starts a viewer session with TCP slow start. With pre-buffering of several seconds, slow start is likely to end before starting playback. Moreover, the startup rebuffering in HTTP-FLV, falling far behind RTM, has always been a painful problem.

Why RTC scenarios did not resolve the problem? In RTC applications, a newly joined participant can send a Full Intra Request (FIR) [17] to the media publisher, enforcing an immediate key video frame to be encoded. That effectively limits the pacing rate by the stream bitrate, and alleviates the startup congestion issue resulting from cached frames. In comparison, there could be hundreds or thousands of viewers per live broadcast, which is orders of magnitude higher than in a typical RTC scenario. Also, the edge CDN server is not the source of the live media content. It would induce ultra-high computation overheads to respond to every FIR from each viewer. For server-side cost control, we choose to disable FIR and stick to periodic GOPs in live streaming, hence the necessity to tailor Vanilla-RTM.

4.3.2 Optimization Solution: Bitrate-Limited Startup. To mitigate startup video rebuffering, we coordinate with our CDN providers to limit the amount of data that edge CDN can aggressively transmit at the start of each viewer session by D_s (in unit of bits). After that, the edge CDN calculates a *bitrate-limited startup pacing rate* for each viewer. To facilitate understanding of the pacing rate calculation, we first explain our GOP cache implementation.

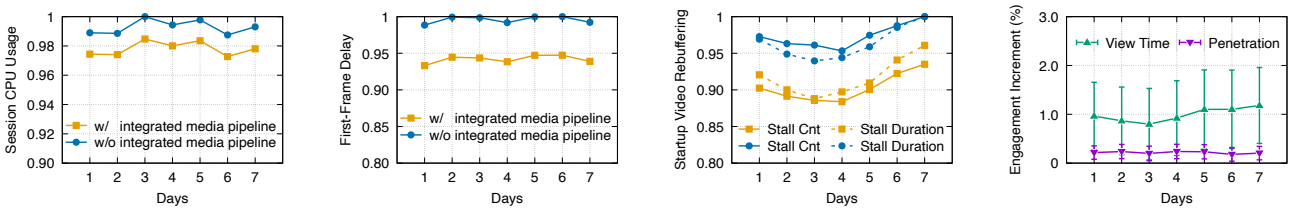
As illustrated in Figure 7, two levels of GOP caches are maintained at each edge CDN node. A single node-level GOP cache always contains the frames from the most recent GOP, *e.g.*, frames of old GOP will be discarded when a new GOP is received from upstream (Figure 7(b)). A per-viewer cache is allocated for each new viewer, initialized by duplicating the node-level cache at the time the viewer request is processed (Figure 7(a)). Any new frame will be appended to all per-viewer caches. Cached frames are only removed from a per-viewer GOP cache after they are transmitted to and acknowledged by the viewer.

Based on the per-viewer GOP cache, the bitrate-limited pacing rate at time t is calculated as,

$$G_m \cdot s_{cache}(t)/d_{cache}(t) \ ,$$

where $s_{cache}(t)$ is the amount of data cached at time t (in unit of bits), and $d_{cache}(t)$ is the playback duration corresponding to $s_{cache}(t)$ (in unit of seconds, calculated based on the number of cached frames). We add a pacing gain G_m (G_m>1) so that the cached frames are transmitted faster than the rate at which new frames are received from upstream. The edge CDN updates the pacing rate on a per-packet basis. Such a bitrate-limited startup phase lasts until there is less than a whole video frame in the viewer GOP cache, which means the session has caught up to the "current". Beyond that time, transmission to the viewer will be naturally limited by the live broadcast bitrate.

Parameter configuration. To determine the value of D_s and G_m, we categorize viewers' access link quality into several tiers, and



(a) Session CPU usage **(b) Video first-frame delay** **(c) Startup video rebuffering** **(d) User engagement**

Figure 8: Sensitivity analysis: integrated media pipeline (blue line with round dots depicts QoE w/o optimization)

find the best configurations for each tier through multiple rounds of large-scale A/B tests. For example, a typical setup for viewers with an ideal access link adopts $D_s = 600$ Kbit and $G_m = 1.1$. At the start of each viewer session, the edge CDN may acquire an estimated access link tier based on a short viewing history of the viewer, *e.g.*, by monitoring average bitrate and loss rate from watching the most recent 10 videos before swiping to this live broadcast.

Then, we leverage the initial aggressively transmitted D_s-bit data to verify the estimated link tier. For example, if the edge CDN detects that a viewer's access link turns into a weak link that should be mapped to a smaller D_s than that of the estimated tier, it will change the configured value of D_s and G_m, and enter the bitrate-limited phase earlier.

We also note that the bitrate-limited pacing rate should be arbitrated by the congestion controller. It is not able to avoid rebuffering in case of insufficient access bandwidth.

4.4 Light-Weight SDP Negotiation

Among the other optimization techniques in RTM, an important one is the adoption of a more light-weight SDP (session description protocol) negotiation. Different from HTTP-FLV where media content transmission can directly follow an HTTP request, the WebRTC stack initiates each session with SDP negotiation, during which the sender and receiver exchange SDP request and answer to agree on various codec-related configurations and features [15]. The negotiation process consumes at least a round trip on the last-mile link, contributing to the first-frame delay degradation in Vanilla-RTM (Table 1) together with the RTC media processing pipeline (§4.1). To mitigate such overhead inherent to WebRTC, we come up with a compressed format for the SDP message, and merge it with other WebRTC handshake messages.

We should point out that light-weight SDP negotiation is not only aimed at RTM-based live streaming. It is actually developed as a general enhancement to our WebRTC stack, and first verified in our RTC services (*e.g.*, Lark).

5 Evaluation

5.1 Sensitivity Analysis

In this section, we first report the results of multiple one-week online A/B tests specific to each optimization mechanism in §4 as a sensitivity analysis. That is followed by the up-to-date overall performance gains of RTM over HTTP-FLV. In the end, to highlight the advantage of low-latency live streaming based on RTM, we share a case study comparing Douyin with 4 other competitor applications during two real-world large-scale sports events.

For metric measurements, our application continuously records client-side QoS and QoE metrics in local logs. Those logs are uploaded to a centralized telemetry system periodically, which are then analyzed, synthesized, and stored in our database. Notably, the presented CPU usage measures the CPU time induced by the whole app process, including kernel usage induced by system calls (but excluding network I/O).

All the A/B tests in this paper involve billions of viewer sessions, and the claimed gains of RTM are statistically significant with a p-value smaller than 5%. Moreover, viewers that watch the same live broadcast may be divided into different tested groups. Thus, we regard them as fair comparisons between the test schemes, with minimized influence of non-technical factors (*e.g.*, different media genres, different device operating systems, *etc.*). We normalize the data with respect to the largest value of each metric.

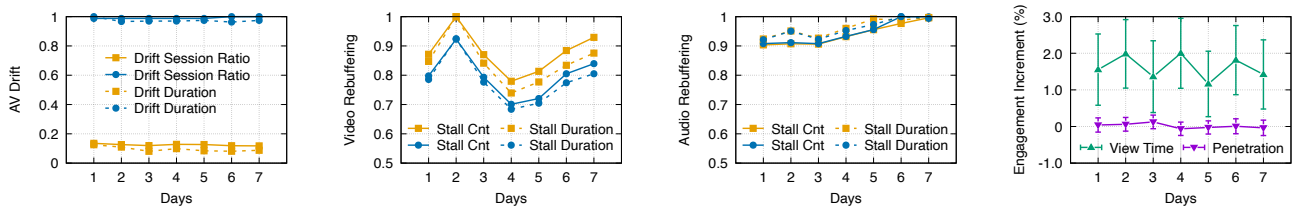
As mentioned in §4.4, the light-weight SDP negotiation was initially developed and tested in our RTC applications. It lowers the first-frame delay of RTM by around 10% after being rolled out in the live streaming scenario, but there is no separate A/B test for that. So the sensitivity analysis focuses on the first three mechanisms in §4.

5.1.1 Integrated Pipeline Alleviates CPU Overhead and Speeds Up First Frame. As illustrated in Figure 8(a)&8(b), two most direct effects of the integrated media processing pipeline are a 1.48% lower CPU usage per viewer session and a 5.4% lower first-frame delay. The reductions come from the elimination of per-session codec initialization and duplicate processing blocks in the RTC and HTTP-FLV media engines.

Besides, we see in Figure 8(c) that the optimized pipeline leads to 7.2% less video rebuffering occurrences and 4.9% shorter rebuffering duration during session startup. We attribute that to the lower CPU pressure with fewer involved media processing blocks, which are mostly implemented in the user space. They are less likely to overwhelm viewer devices upon entering a viewer session.

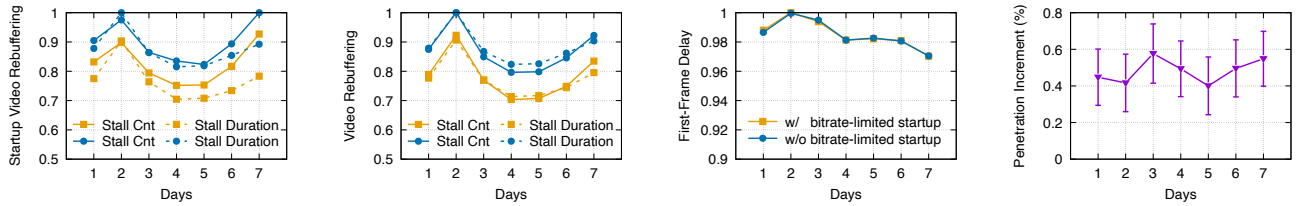
With all the above metrics having top priority for user engagement as analyzed in §3, the integrated pipeline raises the average viewer penetration and daily viewing time by 0.22% and 1.0%, respectively.

5.1.2 Audio-Calibrated Synchronization Reduces AV Drift. The subsequent optimization considered for sensitivity analysis is the audio-calibrated AV synchronization. Unlike the former Integrated Media Pipeline mechanism, the new AV synchronization strategy induces more obvious trade-offs. On one hand, AV drift is significantly reduced of course. To demonstrate that, we present the average per-session drift duration and the ratio of drifted sessions in Figure 9(a).



(a) AV drift (b) Video rebuffering (c) Audio rebuffering (d) User engagement

Figure 9: Sensitivity analysis: audio-calibrated sync (blue line with round dots depicts QoE w/o optimization)



(a) Startup video rebuffering (b) Overall video rebuffering (c) Video first-frame delay (d) Viewer penetration

Figure 10: Sensitivity analysis: bitrate-limited startup (blue line with round dots depicts QoE w/o optimization)

Therein, the drift duration only accounts for the time during which AV drifts exceed the unacceptability thresholds suggested in [16], *i.e.*, audio is ahead of video by more than 90 ms or lags behind by over 185 ms. A session is counted as a drifted session if it experiences such unacceptable drift(s). By calibrating video rendering based on audio, the reduction of drift duration and drifted session ratio are as high as 90.2% and 87.4%.

On the other hand, since the calibration holds back video frame rendering in case of a delayed audio frame, video rebuffering inevitably occurs more often. Each session suffers from 10.2% more video rebuffering instances and 8.4% longer video rebuffering (as in Figure 9(b)). We note that the degradation in rebuffering is only observed for video. As shown in Figure 9(c), the audio rebuffering frequency and duration are not influenced apparently.

Even with the trade-off, the average daily viewing time is still increased by 1.61% (see Figure 9(d)). That does not mean video rebuffering is not important to user engagement. It is because the WebRTC stack and related optimizations in congestion control have managed to achieve such a low rebuffering ratio that AV synchronization becomes more crucial to viewing time in the importance analysis (§3). At the same time, the first video frame is not delayed due to the calibration. So there is no statistically significant degradation in viewer penetration.

5.1.3 Bitrate-Limited Startup Reduces Startup Rebuffering. The main purpose of bitrate-limited startup pacing is to reduce the frequency of video rebuffering in startup phase. As a measure of that, we calculate the average number of video rebuffering instances and the video rebuffering duration in the first 4 seconds of each viewer session. Figure 10(a) shows that the optimization mechanism reduces both the frequency and duration of startup video rebuffering. The number of video rebuffering instances during session startup is reduced by 8.3% on average, and the corresponding rebuffering duration is also reduced by 12.2%. Meanwhile, neither the video first frame nor audio transmission is impacted by the more conservative pacing rate. It achieves almost the same video first-frame delay and audio rebuffering frequency/duration as the original aggressive startup in Vanilla-RTM. We show the first-frame delay in Figure 10(c), but omit the results of audio rebuffering due to limited space.

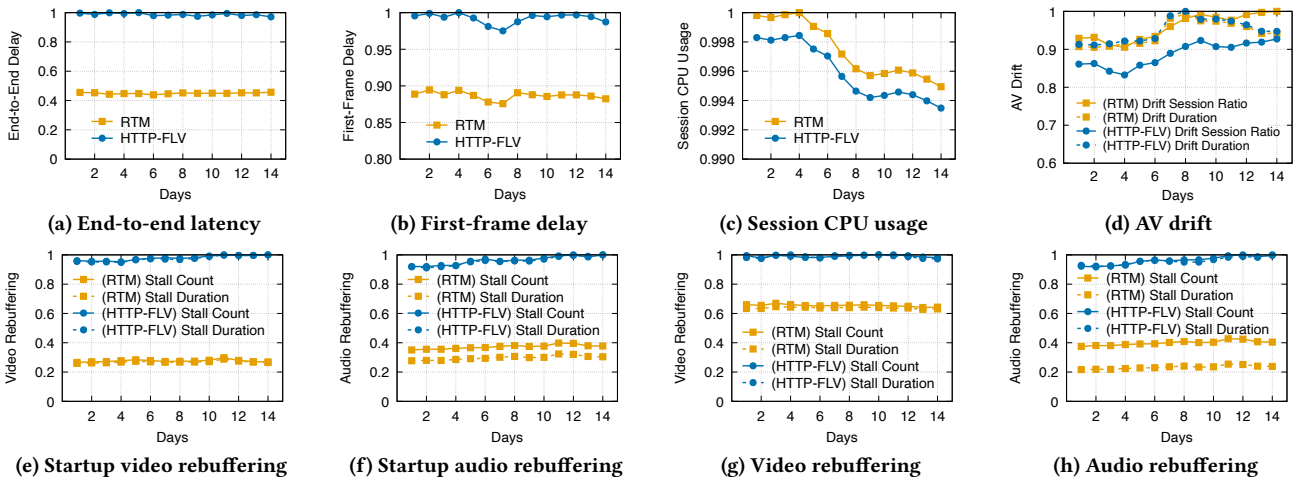
The mitigation of congestion is not only limited to the startup phase. Its benefits extend to the rest of the session time, because the congestion induced during startup usually takes much longer to recover from, especially in weak access networks. As exemplified in Figure 10(b), both the average frequency and duration of video rebuffering per viewer session are reduced, by 10.0% and 12.0%, respectively.

More importantly, bitrate-limited startup pacing ultimately aims at optimized viewer penetration. As expected, it increases our viewer penetration by an average of 0.48%. Although seemingly small, such a penetration gain is confirmed by a 95% statistical significance, and is equivalent to millions more viewers clicking to enter the live broadcast room daily given our tremendous user population.

5.2 RTM *vs.* HTTP-FLV

The sensitivity test results in §5.1 align well with importance analysis in §3. To demonstrate the effectiveness of our optimization methodology more comprehensively, we report a recent 2-week A/B test conducted in 2024Q4 comparing RTM and HTTP-FLV. Figure 11 depicts some key QoE metrics. HTTP-FLV is significantly outperformed by RTM on a majority of metrics. For instance, the average end-to-end latency is reduced by 54.5% (Figure 11(a)). The first-frame delay decreases by 10.6% (Figure 11(b)). In addition, RTM beats HTTP-FLV on all rebuffering-related metrics (Figure 11(e) ~ 11(h)). For example, the startup video rebuffering count shows a considerable gain of 71.8% as in Figure 11(e).

However, the TCP transport still grants some advantages to HTTP-FLV. Compared with the kernel-based TCP protocol stack in HTTP-FLV, RTM's user-space WebRTC stack slightly increases the CPU usage per viewer session by 0.15%. Without the in-order TCP delivery in combination with seconds-long pre-buffering, RTM has an AV drift duration similar to HTTP-FLV, but induces 8.1% more drifted sessions. Fortunately, the optimized CPU usage and



(a) End-to-end latency (b) First-frame delay (c) Session CPU usage (d) AV drift

(e) Startup video rebuffering (f) Startup audio rebuffering (g) Video rebuffering (h) Audio rebuffering

Figure 11: Comprehensive comparisons between RTM and HTTP-FLV

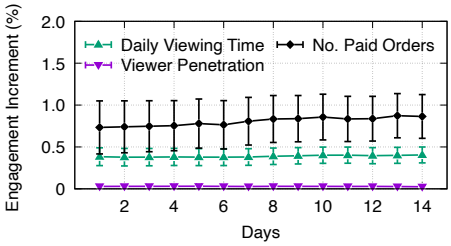


Figure 12: User engagement gains of RTM

AV synchronization in RTM already attract longer viewing time than HTTP-FLV.

To evaluate RTM's gains on user engagement, we further raise the requirement for statistical significance examination to a p-value of 1%. Over the 2-week period, RTM simultaneously optimizes viewer penetration and daily viewing time by 0.03% and 0.39%, respectively (Figure 12). Again, such seemingly small engagement gains are actually quite significant for an already mature service provider like Douyin. They are contributed by at least millions of viewer sessions each day. A strong proof is that the increments in penetration and viewing time convert to 0.8% more paid orders from RTM viewers than in HTTP-FLV. The corresponding revenue gains are in the order of several billions. We do not reveal the actual amount of payments for commercial confidentiality.

We note that HTTP-FLV tested here undergoes years of optimization compared with the version tested with Vanilla-RTM in §2.3.2. RTM's overall performance gains over the up-to-date HTTP-FLV implementation validates the advantage of the WebRTC stack.

5.3 Real-World Case Study

In the end, to illustrate the advantage of RTM more intuitively, we test the end-to-end latency of RTM-based Douyin as well as four other most popular live streaming applications in China, during two large-scale sports events (*i.e.*, FIFA World Cup 2022, and 19th Asian Games in 2023).

Test methodology. While comparing Douyin and another competitor, we open the two applications on two smartphones of identical device brand, with identical operating system version, connected to the same WiFi network, and placed on the same desk. Then, we click on the same match[5] within the two applications, and compare the time difference of seeing the same video frame on the two smartphones. For that purpose, we put a stopwatch besides the smartphones, and record the displayed time and sports game for later analysis. In each trial, we record 10 data points spanning at least 10 minutes. We repeatedly iterate among the four competitors in multiple trials on multiple days during each event, to minimize the influence of various latent factors. The average end-to-end latency differences between Douyin and other competitors are listed in Table 2.

Table 2: E2E latency difference between Douyin and 4 competitors (A negative number means Douyin has lower latency; × denotes the non-rights-holding broadcasters for a event)

	A	B	C	D
FIFA World Cup 2022	−31 s	−3.5 s	×	×
Asian Games 2023	−14 s	×	−4.3 s	−5.8 s

Test results. Under RTM, Douyin achieves an end-to-end latency that is at least 3.5 seconds lower than the other competitors. We even observe dramatic gains of 31 and 14 seconds over Competitor A. According to the dump files, that is because Competitor A adopted media segments with much longer duration, and we inferred it had more conservative pre-buffering. We also notice that from the first event in 2022 to the second one in 2023, competitor A approximately halved its end-to-end latency. We infer that there may be an upgrade in its live streaming architecture between the two events. Yet RTM maintains the leading position of Douyin in terms of live streaming end-to-end latency.

6 Discussion

6.1 Deployment Experiences

Coordination with CDN providers. The full-fledged implementation of RTM requires modifications on the CDN side.

[5]Both events have the same exclusive media rights holder in China. So the live broadcasts of all tested applications are from the same source.

- As explained in §4.3, we conducted multiple rounds of A/B tests while tuning parameters for bitrate-limited startup pacing. In our current deployment, we employ different configurations for different CDN providers in different regions, to adapt to heterogeneous network conditions and CDN edge node distribution.
- With RTM, we still use HTTP-FLV to transmit media contents between CDN origin and edge nodes, because FLV has very low encapsulation overheads. Therefore, when distributing under RTM, an edge CDN node needs to remux the received audio and video frames to RTP packets. In addition, we request the support of new-generation FEC. Those modifications may inevitably induce higher CPU computation overhead. As a result, CDN providers typically charge higher fees for RTM.
- CDN providers are responsible for actual implementation of congestion control. We are not aware of their algorithm details, except in our self-constructed CDN. However, we may discuss with them on how congestion control may be enhanced. For example, we want to maintain a high bitrate, even at the cost of some extent of latency inflation. Correspondingly, some CDN providers, whose congestion control algorithms are based on BBR [18], have deployed more aggressive minimum RTT probing than the original one in BBR.
- We also communicate with our CDN providers on the extensions to the WebRTC stack, such as the support of some self-defined RTCP feedback frames and the lightweight SDP negotiation as discussed in §4.4.

We should point out that our coordination with CDN providers happen bidirectionally. On one hand, we usually validate an optimization scheme on our self-constructed CDN, before issuing a requirement to other 3rd-party providers. On the other hand, we receive many valuable suggestions from 3rd-party providers, as well. For example, the idea of compressing SDP negotiation messages was first contributed by one of our CDN providers.

Operate RTM and HTTP-FLV simultaneously. RTM has been serving our clients for 4 years. As of now, we still have no plans to retire HTTP-FLV in the live streaming scenario, due to the following reasons.

- First, WebRTC-based streaming is not yet universally supported by all CDN providers. Many cost-efficient providers, indispensable for server-side cost control at Douyin, do not support the WebRTC stack at the moment. For viewers served by their servers, HTTP-FLV is our only choice.
- Second, we occasionally observe that RTP/UDP traffic is dropped by some ISPs (Internet service providers) and organizations (*e.g.*, blocked by intentional or problematic firewall configurations). In that case, we have to fall back to HTTP-FLV to guarantee service availability.
- In addition, WebRTC-based media streaming is currently more expensive than HTTP-based distribution [1]. That is mainly because its user-space implementation is more compute-intensive and incurs higher costs on server machines. Ideally, we would like to amortize the cost by fanning out to a large number of viewers per live broadcast. Thus, we may sometimes restrict viewers of some less popular broadcasters to HTTP-FLV. We note that such restriction is per-viewer instead of per-broadcaster. That said, as the number of concurrent viewers specific to a broadcaster surges, subsequent viewers can be directed to RTM in real-time. Additionally, we default all viewers to RTM in some high-value use cases (*e.g.*, e-commerce).

Support for B-frames. B-frame [31] is enabled in many live streaming applications. It has higher compression efficiency, and thus lowers bandwidth requirement and offers better video quality at the same bitrate. However, introducing B-frames to video streams complicates the inter-frame reference relationships, and inevitably increases the encoding and decoding delay. Therefore, the feature is often disabled in RTC scenarios. To match the video quality by HTTP-FLV, we have adapted our RTM media processing pipeline to support B-frames. Although it increases the end-to-end latency[6], that is compensated by the much more significant latency reduction brought by the WebRTC stack. Our support for B-frames comes with many optimizations on unreliable media transmission, *e.g.*, the CDN server and viewer may skip several B-frames to combat bursty packet losses in highly fluctuating networks.

Optimization for ingest. This paper focuses on the streaming protocol on the viewer side, since it is the most critical component from the perspective of end-to-end latency breakdown. In fact, as we mentioned in §1, the complete form of RTM also includes WebRTC-based ingest on broadcaster side. Although out of scope of this paper, we want to strengthen that the significance of ingest protocol optimization is reflected in a different way. Specifically, a single broadcaster may be watched by many concurrent viewers. The delay of a single video frame in the uplink direction may cause rebuffering to hundreds of or even thousands of viewer sessions. Besides, the ingest and streaming protocols are independent of each other in RTM, *i.e.*, a live broadcast ingested by WebRTC may still be delivered to viewers via HTTP-FLV. Correspondingly, to isolate the performance benefits of WebRTC-based ingest, the A/B test results presented in previous sections already filter out live broadcasts ingested by WebRTC.

6.2 Future Direction

QoE modeling. The top-priority metrics to optimize are identified in §3 by an importance analysis aimed at two carefully selected user engagement metrics. Although proven effective in our practical deployment, some important factors may still be overlooked in that process, *e.g.*, interactions between different metrics. On that front, a more comprehensive approach is to construct more complicated and larger QoE model(s) leveraging powerful tools such as deep neural network. Furthermore, we expect that such a QoE model, not necessarily universal in all cases, can be tailored to different user ages and media genres, and help us balance user experience with cost more flexibly. For example, we may adaptively increase the target delay of jitter buffers to fulfill a viewer, if the modeling output tells us that the viewer is likely to tolerate high end-to-end latency, but easily becomes impatient with rebuffering.

Receiver-driven congestion control. Congestion control remains a critical challenge in both academia and industry. Through our collaboration with multiple third-party CDNs, we have observed significant disparities in their congestion control capabilities. To

[6]The actual latency increment depends on the GOP structure, *e.g.*, around 200 ms with 3 B-frames between two consecutive I/P-frames

address this issue, we plan to explore a receiver-driven mechanism where the client side estimates available bandwidth and feeds that back to the server side. By combining the sender-side and receiver-side bandwidth estimation, more accurate congestion control strategies may be derived. Such a mechanism enables more consistent user experiences via different CDNs. Ultimately, there is a chance that CDNs can offload the core congestion control logic to the receiver side (at least partially).

Unified architecture and protocols. Live streaming is one of several major services we provide at ByteDance. At the moment, different services adopt different application-layer and transport-layer protocols, and run on different architectures. For example, the server-side media forwarding in live streaming is still based on TCP, while it is end-to-end WebRTC in RTC scenarios such as collaborative streaming. That is not ideal from the perspective of operation and maintenance complexity. We already have some preliminary attempts to unify the publish-side architecture of live streaming and collaborative streaming [38]. In the future, we are interested to further unify more services (*e.g.*, on-demand video streaming), and extend to unified media transport protocols (*e.g.*, leveraging the idea of MOQ [41]).

7 Related Work

Live streaming protocols. MPEG DASH (MPEG Dynamic Adaptive Streaming over HTTP) [49] and HLS (HTTP Live Streaming) [43] are two representative protocols for on-demand video streaming and live streaming. However, they can only achieve tens of seconds of end-to-end latency, which are not enough to fulfill users' ever-increasing requirements for real-time interaction during live streaming. Aimed at low latency scenarios, LL-DASH [26] are LL-HLS [11] are then proposed. By introducing smaller and partial chunks, they manage to reduce the end-to-end latency to a few seconds. Other protocols such as RTMP [10] and HTTP-FLV, based on the FLV (Flash Video) format [9], have comparable end-to-end latency. Beyond that, to fulfill more stringent latency requirements, WebRTC [5] is invented to support RTC applications (*e.g.*, multi-party video conferencing [35, 37]). By integrating WebRTC into the live streaming scenario, pioneering CDN providers have started to offer low-latency live streaming services [33, 36, 45, 55]. This work is built on that basis, and holds the perspective of an application service provider.

QoS and QoE Modeling. QoS/QoE models capture how QoS metrics determine QoE metrics, and how they influence user experience. They have been extensively utilized to optimize video streaming [14, 22, 28, 40, 53, 54, 56, 58, 59]. Some of the existing works aim at optimizing an empirical objective function, computed as a weighted sum of several common QoS metrics [53, 54]. Some others leverage either crowdsourcing user studies [28, 58] or massive collected traces from real-world deployment [56] to fit or learn how different metrics can be mapped to subjective user satisfaction and engagement. A majority of those works resolve bitrate adaptation in on-demand video streaming. Our importance analysis with respect to user engagement belongs to this area, with a focus on live streaming. Unlike these prior works, we prioritize two key metrics (viewer penetration and daily viewing time), which are specific to the live streaming scenario as well as our massive user population. In the future, we plan to explore more complex models for more tailored insights to individual users.

Transport layer optimization. Another category related and complementary to this paper is the transport layer optimizations. Therein, congestion control attracts the most interest. Various congestion control algorithms are proposed, ranging from rule-based algorithms [13, 18, 20] to more recent learning-based ones [8, 51, 52, 57]. Besides, multi-path transport has been an important extension, especially for the first/last-mile, where it can combine the bandwidth of multiple access links to improve resistance against network fluctuations [27, 42, 62]. In addition, loss recovery mechanisms are designed to recover lost media data by timely retransmissions and/or adding redundancy [32, 39, 44] . Further optimizations focus on the collaborations between the transport layer and video encoding layer [28, 34, 35, 61]. For example, Mamba [34] designed a multi-agent solution to control multiple encoding parameters based on observations from both layers. Thanks to existing literature on above, we manage to achieve extremely low video rebuffering ratios, and thus, are able to focus on some less recognized metrics (such as device CPU overhead) in this paper.

8 Conclusion

In this paper, we share our experiences of taming WebRTC as the streaming protocol in RTM, the low-latency live streaming system of Douyin. Guided by an importance analysis targeting viewer penetration and daily viewing time, we propose various mechanisms to optimize 4 top-priority QoE metrics (*i.e.*, first-frame delay, startup video rebuffering count, AV drift, and per-session CPU usage). After 4 years of practical deployment, RTM is serving billions of live streaming viewers daily. Compared with the previously deployed HTTP-FLV streaming protocol, RTM contributes to 0.03% and 0.39% increments in viewer penetration and viewing time, respectively, corresponding to millions more viewers entering a live broadcast room and millions more hours of viewing time each day. The optimized user engagement further increases the number of paid orders from viewers by 0.8%.

We hope our insights from deploying WebRTC as the streaming protocol in RTM can inspire further research, *e.g.*, more tailored and customized QoE modeling that accommodate differences in users and media contents.

Acknowledgments

The authors would like to thank the shepherd Sadjad Fouladi and the anonymous reviewers for their insightful comments and valuable suggestions. We are also grateful to all the teams at ByteDance who contributed to the deployment of RTM, especially Lei Wang and Qizheng Wang from the LiveCore Team. Junfeng Yang's work was partially supported by Natural Science Foundation of Hunan Province (No. 2024JJ5113). Lei Zhang's work was partially supported by National Natural Science Foundation of China (No. 62272317) and Shenzhen Science and Technology Program (No. JCYJ20220531103402006). Zhi Wang's work was partially supported by National Key Research and Development Project of China (No. 2023YFF0905502), and National Natural Science Foundation of China (No. 92467204 and 62472249).

References

[1] Alibaba ApsaraVideo Live Pricing. https://www.alibabacloud.com/en/product/apsaravideo-for-live/pricing. Accessed: (2025.1).
[2] Douyin Live. https://live.douyin.com/. Accessed: (2025.1).
[3] Lark. https://www.larksuite.com/. Accessed: (2025.1).
[4] Twitch. https://www.twitch.tv/. Accessed: (2025.1).
[5] WebRTC. https://webrtc.github.io/webrtc-org/architecture/. Accessed: (2025.1).
[6] WebRTC Codebase. https://webrtc.googlesource.com/src/. Accessed: (2025.1).
[7] YouTube Live. https://www.youtube.com/live. Accessed: (2025.1).
[8] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. 2020. Classic meets modern: A pragmatic learning-based congestion control for the internet. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 632–647.
[9] Adobe. 2010. Adobe Flash Video File Format Specification. https://download.macromedia.com/f4v/video_file_format_spec_v10_1.pdf. Accessed: (2025.1).
[10] Adobe. 2012. Adobe RTMP Specification. https://rtmp.veriskope.com/docs/spec/. Accessed: (2025.1).
[11] Apple. Enabling Low-Latency HTTP Live Streaming (HLS). https://developer.apple.com/documentation/http-live-streaming/enabling-low-latency-http-live-streaming-hls. Accessed: (2025.1).
[12] Apple. HTTP Live Streaming (HLS) authoring specification for Apple devices. https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices. Accessed: (2025.1).
[13] Venkat Arun and Hari Balakrishnan. 2018. Copa: Practical {Delay-Based} congestion control for the internet. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 329–342.
[14] Athula Balachandran, Vyas Sekar, Aditya Akella, Srinivasan Seshan, Ion Stoica, and Hui Zhang. 2013. Developing a predictive model of quality of experience for internet video. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 339–350.
[15] Ali C. Begen, Paul Kyzivat, Colin Perkins, and Mark J. Handley. 2021. SDP: Session Description Protocol. RFC 8866. https://doi.org/10.17487/RFC8866
[16] ITU-R Recommendation BT. 1998. 1359-1, Relative timing of sound and vision for broadcasting. *International Telecommunication Union-Radiocommunication Sector* (1998).
[17] Bo Burman, Stephan Wenger, Magnus Westerlund, and Umesh Chandra. 2008. Codec Control Messages in the RTP Audio-Visual Profile with Feedback (AVPF). RFC 5104. https://doi.org/10.17487/RFC5104
[18] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. 2017. BBR: Congestion-based congestion control. *Commun. ACM* 60, 2 (2017), 58–66.
[19] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2016. Analysis and design of the google congestion control for web real-time communication (WebRTC). In *ACM MMSys*. 1–12.
[20] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2017. Congestion control for web real-time communication. *IEEE/ACM Transactions on Networking* 25, 5 (2017), 2629–2642.
[21] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
[22] Sheng Cheng, Han Hu, Xinggong Zhang, and Zongming Guo. 2023. Rebuffering but not suffering: Exploring continuous-time quantitative qoe by user's exiting behaviors. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
[23] Yusuf Cinar, Peter Pocta, Desmond Chambers, and Hugh Melvin. 2021. Improved jitter buffer management for WebRTC. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)* 17, 1 (2021), 1–20.
[24] Tencent Cloud. 2024. Cloud Streaming Services. https://www.tencentcloud.com/products/css. Accessed: (2025.1).
[25] The Khang Dang, Nitinder Mohan, Lorenzo Corneo, Aleksandr Zavodovski, Jörg Ott, and Jussi Kangasharju. 2021. Cloudy with a chance of short RTTs: analyzing cloud connectivity in the internet. In *ACM IMC*. 62–79.
[26] DASH-IF. 2017. Low-Latency DASH Modes. https://dashif.org/news/low-latency-dash/. Accessed: (2025.1).
[27] Sandesh Dhawaskar Sathyanarayana, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2023. Converge: Qoe-driven multipath video conferencing over webrtc. In *ACM SIGCOMM*. 637–653.
[28] Sadjad Fouladi, John Emmons, Emre Orbay, Catherine Wu, Riad S Wahby, and Keith Winstein. 2018. Salsify:{Low-Latency} network video through tighter integration between a video codec and a transport protocol. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 267–282.
[29] Ingemar Johansson and Zaheduzzaman Sarker. 2017. Self-Clocked Rate Adaptation for Multimedia. RFC 8298. https://doi.org/10.17487/RFC8298
[30] Ari Keränen, Christer Holmberg, and Jonathan Rosenberg. 2018. Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal. RFC 8445. https://doi.org/10.17487/RFC8445
[31] Ravi Krishnamurthy, John W Woods, and Pierre Moulin. 1999. Frame interpolation and bidirectional prediction of video using compactly encoded optical-flow fields and label fields. *IEEE transactions on circuits and systems for video technology* 9, 5 (1999), 713–726.
[32] Insoo Lee, Seyeon Kim, Sandesh Sathyanarayana, Kyungmin Bin, Song Chong, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2022. R-FEC: RL-based FEC Adjustment for Better QoE in WebRTC. In *Proceedings of the 30th ACM International Conference on Multimedia*. 2948–2956.
[33] Jinyang Li, Zhenyu Li, Ri Lu, Kai Xiao, Songlin Li, Jufeng Chen, Jingyu Yang, Chunli Zong, Aiyun Chen, Qinghua Wu, et al. 2022. LiveNet: a low-latency video transport network for large-scale live streaming. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 812–825.
[34] Yueheng Li, Zicheng Zhang, Hao Chen, and Zhan Ma. 2023. Mamba: Bringing Multi-Dimensional ABR to WebRTC. In *Proceedings of the 31st ACM International Conference on Multimedia*. 9262–9270.
[35] Xianshang Lin, Yunfei Ma, Junshao Zhang, Yao Cui, Jing Li, Shi Bai, Ziyue Zhang, Dennis Cai, Hongqiang Harry Liu, and Ming Zhang. 2022. GSO-simulcast: global stream orchestration in simulcast video conferencing systems. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 826–839.
[36] Zhejiayu Ma, Soufiane Rouibia, Frédéric Giroire, and Guillaume Urvoy-Keller. 2022. Neighbor Selection Strategies in the Wild for CDN/V2V WebRTC Live Streaming: Can we learn what a good neighbor is?. In *IEEE LCN*. IEEE, 295–298.
[37] Tong Meng, Wenfeng Li, Chao Yuan, Changqing Yan, and Le Zhang. 2025. AsTree: An Audio Subscription Architecture Enabling Massive-Scale Multi-Party Conferencing. In *USENIX NSDI*. 653–666.
[38] Tong Meng, Wei Zhang, Dong Chen, Zhen Wang, Quanqing Li, Changqing Yan, Wei Yang, Chao Yuan, Le Zhang, Jianxin Kuang, et al. 2025. AnchorNet: Bridging Live and Collaborative Streaming with a Unified Architecture. In *USENIX ATC*.
[39] Zili Meng, Xiao Kong, Jing Chen, Bo Wang, Mingwei Xu, Rui Han, Honghao Liu, Venkat Arun, Hongxin Hu, and Xue Wei. 2024. Hairpin: Rethinking packet loss recovery in edge-based interactive video streaming. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 907–926.
[40] Gabriel Mittag, Babak Naderi, Vishak Gopal, and Ross Cutler. 2023. LSTM-Based Video Quality Prediction Accounting for Temporal Distortions in Videoconferencing Calls. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
[41] Suhas Nandakumar, Victor Vasiliev, Ian Swett, and Alan Frindell. 2025. *Media over QUIC Transport*. Internet-Draft draft-ietf-moq-transport-13. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-moq-transport/13/ Work in Progress.
[42] Yunzhe Ni, Zhilong Zheng, Xianshang Lin, Fengyu Gao, Xuan Zeng, Yirui Liu, Tao Xu, Hua Wang, Zhidong Zhang, Senlang Du, et al. 2023. CellFusion: Multipath Vehicle-to-Cloud Video Streaming with Network Coding in the Wild. In *ACM SIGCOMM*. 668–683.
[43] Roger Pantos and William May. 2017. *HTTP live streaming*. Technical Report.
[44] Yi Qiao, Han Zhang, and Jilong Wang. 2024. NetFEC: In-network FEC Encoding Acceleration for Latency-sensitive Multimedia Applications. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2348–2357.
[45] Ishani Sarkar, Soufiane Rouibia, Dino Lopez-Pacheco, and Guillaume Urvoy-Keller. 2020. Proactive information dissemination in webrtc-based live video distribution. In *IEEE IWCMC*. IEEE, 304–309.
[46] Henning Schulzrinne, Stephen L. Casner, Ron Frederick, and Van Jacobson. 2003. RTP: A Transport Protocol for Real-Time Applications. RFC 3550. https://doi.org/10.17487/RFC3550
[47] M Scott, Lee Su-In, et al. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017), 4765–4774.
[48] Rohit Shewale. 2023. 42 Live Streaming Statistics For 2024 (Data & Insights). https://fashionhunters.live/fr/blogs/blog/42-live-streaming-statistics-for-2024-data-insights. Accessed: (2025.1).
[49] ISO Standard. 2014. Dynamic adaptive streaming over HTTP (DASH)-Part 1: Media presentation description and segment formats. *ISO/IEC* (2014), 23009–1.
[50] Streaming Video Technology Alliance (SVTA). Low Latency Streaming. https://www.svta.org/working-group/low-latency-streaming/. Accessed: (2025.1).
[51] Zhengxu Xia, Yajie Zhou, Francis Y Yan, and Junchen Jiang. 2022. Genet: automatic curriculum generation for learning adaptation in networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 397–413.
[52] Chen-Yu Yen, Soheil Abbasloo, and H Jonathan Chao. 2023. Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 255–274.
[53] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 325–338.
[54] Anlan Zhang, Chendong Wang, Bo Han, and Feng Qian. 2022. {YuZu}:{Neural-Enhanced} volumetric video streaming. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 137–154.
[55] Huanhuan Zhang, Congkai An, Anfu Zhou, Yifan Zhu, Weilin Sun, Yixuan Lu, Jiahao Chen, Liang Liu, Huadong Ma, and Aiguo Fei. 2024. Venus: Enhancing

QoE of Crowdsourced Live Video Streaming by Exploiting Multiflow Viewer Assistance. In *ACM MobiCom*. 170–184.

[56] Haodan Zhang, Yixuan Ban, Zongming Guo, Zhimin Xu, Qian Ma, Yue Wang, and Xinggong Zhang. 2023. QUTY: Towards Better Understanding and Optimization of Short Video Quality. In *Proceedings of the 14th Conference on ACM Multimedia Systems*. 173–182.

[57] Wei Zhang, Xuefeng Tao, and Jianan Wang. 2024. NAORL: Network Feature Aware Offline Reinforcement Learning for Real Time Bandwidth Estimation. In *ACM MMSys*. 326–331.

[58] Xu Zhang, Yiyang Ou, Siddhartha Sen, and Junchen Jiang. 2021. {SENSEI}: Aligning Video Streaming Quality with Dynamic User Sensitivity. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 303–320.

[59] Zejun Zhang, Xiao Zhu, Anlan Zhang, and Feng Qian. 2024. An In-depth Study of Bandwidth Allocation across Media Sources in Video Conferencing. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 7696–7704.

[60] Yuankang Zhao, Qinghua Wu, Gerui Lv, Furong Yang, Jiuhai Zhang, Feng Peng, Yanmei Liu, Zhenyu Li, Ying Chen, Hongyu Guo, et al. 2024. JitBright: towards Low-Latency Mobile Cloud Rendering through Jitter Buffer Optimization. In *ACM NOSSDAV*. 36–42.

[61] Anfu Zhou, Huanhuan Zhang, Guangyuan Su, Leilei Wu, Ruoxuan Ma, Zhen Meng, Xinyu Zhang, Xiufeng Xie, Huadong Ma, and Xiaojiang Chen. 2019. Learning to coordinate video codec with transport protocol for mobile video telephony. In *The 25th Annual International Conference on Mobile Computing and Networking*. 1–16.

[62] Yuhan Zhou, Tingfeng Wang, Liying Wang, Nian Wen, Rui Han, Jing Wang, Chenglei Wu, Jiafeng Chen, Longwei Jiang, Shibo Wang, et al. 2024. AUGUR: Practical Mobile Multipath Transport Service for Low Tail Latency in Real-Time Streaming. In *USENIX NSDI*. 1901–1916.